

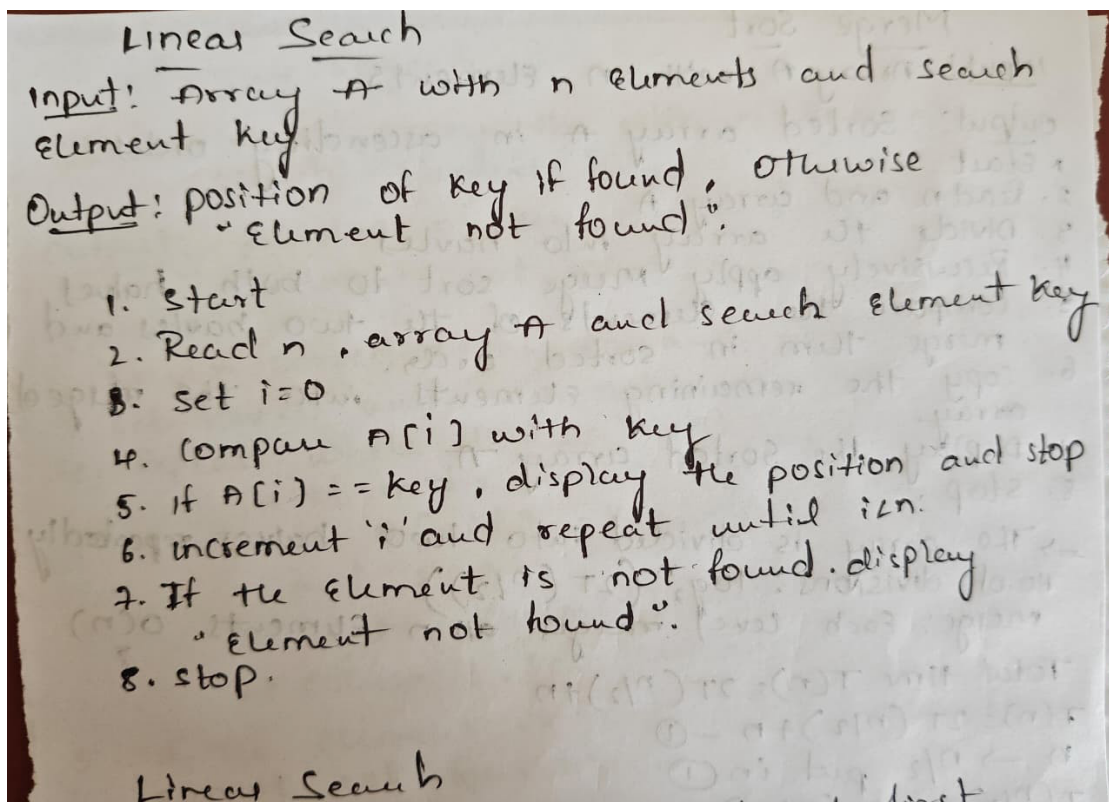


PRACTICAL: 02

Aim: To implement and find the time analysis of Linear Search and Binary Search algorithm.

A) Linear Search:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

int main() {

    int arr[100],i,n,key;

    cout<<"enter number of array elements:"<<endl;

    cin>>n;

    cout<<"enter array elements:"<<endl;

    for(i=0;i<n;i++){

        cin>>arr[i];

    }

    cout<<"enter element to serach:"<<endl;

    cin>>key;

    for(i=0;i<n;i++){

        if(arr[i]==key)

        {

            cout<<"element found at index of:"<<i;

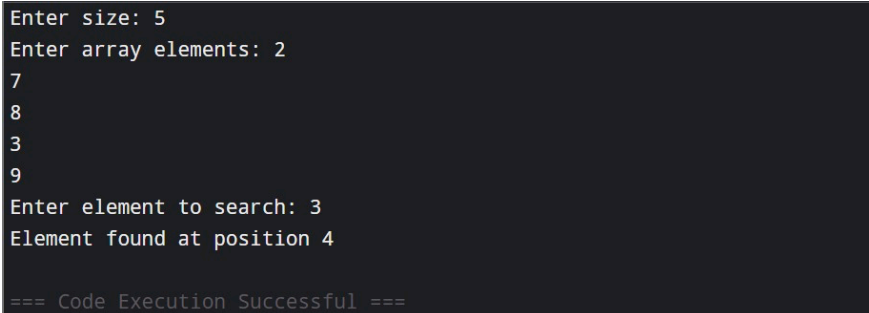
        }

    }

    return 0;

}
```

Output:



```
Enter size: 5
Enter array elements: 2
7
8
3
9
Enter element to search: 3
Element found at position 4

=== Code Execution Successful ===
```



Time Complexity:

Linear Search
Best case : $O(1)$ - Element at first
Average case : $O(n)$
Worst case : $O(n)$ - Element is last (or)
not found.
 $\Rightarrow$ In worst case it checks n elements
 $\therefore T(n) = O(n)$

B) Binary Search:

Algorithm:

Binary Search
Input : Sorted array A with n elements and
search element key
Output : position of key if found, otherwise
"element not found"
1. Start
2. Read n , sorted array A and search element key
3. set $low = 0$ and $high = n - 1$
4. Repeat while $low \leq high$:
 $\rightarrow$ set $mid = (low + high) / 2$
 $\rightarrow$ if $A[mid] == key$, display the position
 and stop
 $\rightarrow$ if $key < A[mid]$, set $high = mid - 1$
 $\rightarrow$ otherwise, set $low = mid + 1$
5. if the element is not found, display
 "element not found"
6. stop.
 $\Rightarrow$ Binary search only one half $\rightarrow T(n/2)$



Program / Code:

```
#include <iostream>

using namespace std;

void bubblesort(int arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n-1; while(low <=
    high) {
        int mid = (low + high) / 2; if(arr
        [mid] == key) return mid; else if(arr
        [mid] < key) low = mid+1; else high
        = mid-1;
    }
    return -1;
}

int main() { int arr[100], n, key; cout <<
"Enter number of array elements: ";
```

```
cin >> n;

cout << "Enter array elements: ";
for(int i=0; i<n; i++) {
    cin >> arr[i];
}

bubblesort(arr, n);
cout << "Sorted elements: ";
for(int i=0; i<n; i++) {
    cout << arr[i] << " ";
}
cout << endl;
cout << "Enter element to search: ";
cin >> key;
int result = binarySearch(arr, n, key);
if(result != -1)
    cout << "Element found at index " << result << endl;
else
    cout << "Element not found" << endl;
return 0;
}
```

Output:

```
Enter size: 5
Enter sorted array elements: 2
7
13
22
31
Enter element to search: 22
Element found at position 4

=== Code Execution Successful ===
```

Time Complexity:

$\Rightarrow$ Binary search only one half $\rightarrow T(n/2)$
 $T(n) = T(n/2) + 1$ — (1)
 $n \rightarrow \frac{n}{2}$ put in (1)
 $T(n/2) = T(n/4) + 1$ — (2)
 sub (2) in (1)
 $T(n) = T(n/4) + 2$ — (3)
 $n \rightarrow \frac{n}{4}$ put in (3)
 $T(n/4) = T(n/8) + 3$
 $\vdots$
 $T(n) = T(n/2^k) + k$
 $\frac{n}{2^k} = 1, n = 2^k$
 Apply log on b.s
 $\log n = \log 2^k$
 $k = \log n$
 $T(n) = T\left(\frac{n}{2^{\log n}}\right) + \log n$
 $\Rightarrow T\left(\frac{n}{n^{\log 2}}\right) + \log n$
 $= T(1) + \log n$
 $\therefore T(n) = O(\log n)$

worst case: $O(\log n)$
 avg case: $O(\log n)$
 Best case: $O(1)$

Conclusion:

In this practical, we studied and implemented two searching algorithms: Linear Search checks each element one by one, so it takes more time as the list size grows. Binary Search is much faster because it works only on sorted lists and cuts the list in half every time, making it a better choice for large data.